Pruning and Int8 Quantization for CNN Inference on a Cortex-M7 Microcontroller: A Controlled Multi-Seed Study

Yerman Bekzat
Astana IT University
Astana, Kazakhstan
yermanbekzat@gmail.com

***Abstract*—Reported gains from quantization and pruning on embedded hardware are hard to compare: studies differ in models, platforms and metrics, and often report energy or latency without a matching accuracy analysis. We present a controlled comparison under one fixed, pre-registered protocol. MobileNetV2 trained on CIFAR-10 is pruned structurally at 0, 25, 50 and 75 percent of its channels and quantized to int8 with post-training quantization and quantization-aware training, giving 48 models over three random seeds. Accuracy is measured on the exported models and latency and memory on an STM32H7B3 microcontroller running at 280 MHz. Int8 quantization alone reduces latency and estimated energy by 4.2 times at a cost of 1.6 percentage points of accuracy; adding 25 percent pruning raises the saving to 5.1 times at a cost of 3.6 points. No compressed configuration meets the pre-registered budget of 1 point. Quantization-aware training beats post-training quantization by 0.4 points on average, and its advantage grows with the pruning ratio. Multiply-accumulate counts rank latency correctly within one numeric precision but not across precisions. At 50 to 75 percent pruning, int8 accuracy varies by up to 33 points across seeds, against at most 2.3 points for full-precision models, so single-seed results can misrepresent aggressive compression. Code, logs and results are public.**

***Index Terms*—TinyML, quantization, structured pruning, microcontrollers, energy efficiency, benchmarking**

I. Introduction

Running convolutional neural networks (CNNs) on the device that collects the data removes the latency and energy cost of sending it to the cloud [1]. On microcontrollers the obstacle is size: even compact CNNs need millions of multiply-accumulate operations (MACs) and hundreds of kilobytes of memory. Quantization and pruning are the two main levers for closing that gap [2], [3], often combined with hardware–software co-design [4]–[6].

The evidence on how far these levers go is hard to use. In a review of the literature on energy-efficient CNN acceleration for embedded systems we found three recurring problems. First, studies use different datasets, platforms and metrics, so their figures cannot be compared [7], [8]. Second, energy and latency gains are often reported without a rigorous account of the accuracy lost [9]; reports even disagree on whether int8 quantization costs accuracy at all [9], [10]. Third, techniques are often evaluated without modeling memory bandwidth and resource limits, so gains estimated from proxies such as operation counts may not appear on a real device [10], [11].

This paper addresses these problems with a small, controlled study in which only the compression settings change. Its contributions are:

- A fixed protocol: one model, one dataset, one microcontroller and one measurement procedure, with the plan, hypotheses and selection rule committed before any experiment and every deviation logged.
- A full factorial comparison of structured pruning (four ratios) and int8 quantization (post-training and quantization-aware) over three seeds, 48 models in total, with accuracy measured on the deployed model files and latency and memory measured on an STM32H7B3 board.
- Three findings for practitioners and reviewers: int8 dominates pruning alone on the accuracy–energy front; MAC counts mislead across numeric precisions; and int8 accuracy after aggressive pruning varies so much across seeds that single-seed reports are unreliable.
- An open archive of code, configurations, raw device logs and results [12].

II. Related Work

Quantization. Integer-only int8 inference with per-channel weights and per-tensor activations is the standard deployment path for CNNs [3], [13]. On microcontrollers, mixed low-precision schemes fit larger networks into memory [14], and instruction-set extensions speed up quantized kernels on RISC-V cores [15]. Depthwise-separable networks such as MobileNet are known to be sensitive to per-tensor quantization because channel ranges differ widely [13], [16], and quantization combined with fast convolution algorithms can be numerically unstable [17].

Pruning. Pruning moved from removing individual weights by magnitude [18] to removing whole filters by their norm [19] and to energy-aware selection [11]. Structured pruning yields a smaller dense network that any hardware can run faster, whereas unstructured sparsity needs dedicated support. Dependency-aware tools now prune architectures with residual connections automatically [20]. Pruning and quantization have been combined since Deep Compression [2].

Embedded deployment and benchmarking. Optimized kernels such as CMSIS-NN make int8 inference practical on Cortex-M cores [21], and MLPerf Tiny defines a common benchmark for such devices [22]. Accelerator studies cut energy on ASICs and FPGAs, mainly by reducing data movement [4], [7]; approximate computing trades accuracy for energy [23], and adaptive schemes tune that trade-off at run time [24], [25]. Critiques of the pruning literature point to inconsistent baselines and metrics [8], and work on benchmark variance shows that ignoring seed-to-seed variation can lead to wrong conclusions [26]. Our study applies these lessons to compression on a real microcontroller.

III. Method

A. Design and hypotheses

The study crosses pruning ratio (0, 25, 50, 75 % of channels) with numeric format (fp32, int8 post-training quantization (PTQ), int8 quantization-aware training (QAT)), each trained with seeds 1–3. Model, data, training recipe, board and measurement are fixed. The plan stated three hypotheses: H1, pruning combined with int8 dominates either technique alone on the accuracy–energy Pareto front; H2, QAT loses less accuracy than PTQ, and more so at high pruning; H3, MAC count preserves the ranking of configurations by measured latency (Spearman $\rho \geq 0.8$). It also fixed a selection rule: the best configuration is the one with the lowest energy whose mean accuracy is within 1 pp of the fp32 baseline.

B. Model and training

We use MobileNetV2 [27] with width multiplier 0.35 on CIFAR-10 [28]. For 32×32 inputs the first convolution and the first stride-2 depthwise convolution use stride 1, so the last feature map is 4×4. The baseline has 408,938 parameters and 17.6 M MACs. Each seed trains its own baseline in PyTorch [29] for 100 epochs with SGD (Nesterov momentum 0.9, learning rate 0.05 with cosine decay, weight decay 4×10^{-5}, batch 128), random crops and horizontal flips.

C. Structured pruning

Channels are removed with Torch-Pruning [20], which keeps channels coupled by residual connections consistent. Importance is the L1 norm of each channel's weights [19]; the same ratio is applied to every layer except the classifier, so all seeds of a ratio share one architecture (Table II). Each pruned model is fine-tuned for 15 epochs (learning rate 0.01, cosine decay).

D. Quantization

Models are exported to ONNX (opset 13, batch normalization folded) and quantized with ONNX Runtime [30] static quantization in QDQ format: int8 weights, symmetric per channel; int8 activations, asymmetric per tensor; 500 fixed calibration images. The activation calibration method was chosen on a validation split of 5,000 training images disjoint from the calibration set, never on the test set. Candidates were min–max and percentile calibration (99.999, 99.99, 99.9); the fixed rule picks the smallest mean accuracy drop over the 12 fp32 models, with ties within 0.1 pp going to the simpler method. Percentile 99.99 won with a mean drop of 6.71 pp against 7.96 pp for min–max; min–max was as good at 0–25 % pruning, and the difference arose at 50–75 % (11.27 vs. 6.77 pp at 50 %).

For QAT, batch normalization is folded into the convolutions and training rounds weights (per channel) and activations (after each ReLU6, after each block's projection convolution and after each residual addition) to int8 in the forward pass with a straight-through gradient. Activation ranges are computed on the same 500 calibration images with the same percentile rule and refreshed before every epoch, so training sees the ranges the deployed model will receive. Afterwards the weights are left on the int8 grid and the model passes through the same quantizer as PTQ; PTQ and QAT models therefore have identical graphs and differ only in weight values. QAT uses learning rate 0.001 with cosine decay for 5 epochs (QAT-5, pre-registered). As a sensitivity check we added 15 epochs (QAT-15), the budget of the pruning fine-tune, both starting from the same fp32 model.

E. Hardware and metrics

Latency and memory are measured on an STM32H7B3I-DK board (Cortex-M7, 280 MHz, 2 MB Flash, 1.4 MB RAM) through ST Edge AI Developer Cloud with ST Edge AI Core 4.1.0 [31]. The models are uploaded already quantized. The code-generation objective is Balanced, except for the 25 % int8 models, which only link with the RAM objective. Weights reside in internal Flash and activations in internal RAM for every model. Latency is the on-device duration of one inference; Flash is weights plus runtime, RAM is activations plus runtime. ST's validation compares the board output with the ONNX model on a random input: cosine similarity is 1.0 for all models but one (0.9987, 75 % QAT).

Seeds change weight values but not layer sizes, so seed-1 models represent each architecture on the board. PTQ and QAT latencies agree within 0.1 %, and int8 uses their mean. Energy per inference is $E = P\,t$ with $P = 3.3\,\mathrm{V} \times 120\,\mu\mathrm{A/MHz} \times 280\,\mathrm{MHz} = 110.9\,\mathrm{mW}$, the typical run-mode figure for this device [32]. It is an estimate for the microcontroller alone and is proportional to latency. Accuracy is top-1 on the 10,000 CIFAR-10 test images, computed with ONNX Runtime on the exported file.

F. Statistics

We report mean $\pm$ standard deviation over seeds. For H2, differences are paired by pruning ratio and seed; we give 95 % t-intervals per ratio and, over all 12 pairs, a one-sided Wilcoxon signed-rank test and a Spearman correlation with the pruning ratio. H3 uses Spearman's ρ over the eight distinct architectures.

G. Deviations from the plan

All deviations are dated in the project log [12]. (i) The planned NUCLEO-H743ZI2 board could not link the fp32

TABLE I
TOP-1 ACCURACY (%), MEAN ± STD OVER 3 SEEDS

Pruned	fp32	PTQ	QAT-5	QAT-15
0 %	89.49 ± 0.13	87.88 ± 0.68	87.71 ± 0.88	87.94 ± 0.65
25 %	87.60 ± 0.43	85.81 ± 0.81	85.88 ± 0.74	86.08 ± 0.73
50 %	81.64 ± 0.15	75.00 ± 5.92	75.66 ± 6.00	77.14 ± 4.67
75 %	66.97 ± 1.21	52.01 ± 17.75	53.19 ± 16.88	54.14 ± 15.45

TABLE II
MEASURED COST PER ARCHITECTURE ON STM32H7B3 (280 MHz). ENERGY IS ESTIMATED AS 110.9 mW × LATENCY

Pruned	Params	MACs (M)	Format	Latency (ms)	Flash (KB)	RAM (KB)	Energy (mJ)
0 %	408,938	17.55	fp32	427.2	1,591	273	47.36
			int8	100.5	475	137	11.14
25 %	239,554	10.77	fp32	289.9	936	221	32.15
			int8	84.2	297	84	9.33
50 %	114,938	5.58	fp32	165.4	457	151	18.34
			int8	42.3	167	42	4.69
75 %	35,090	1.99	fp32	70.0	151	115	7.77
			int8	20.2	79	35	2.24

baseline, because its benchmark firmware places activations in 128 KB of tightly coupled memory; an STM32N6 board was rejected because it has a neural accelerator and no internal Flash. (ii) A first run used min–max calibration, and QAT used moving-average ranges discarded at export. Int8 accuracy collapsed at 50–75 % pruning, and QAT was no better than PTQ. We replaced both with the procedures above and archived the first run. (iii) QAT-15 was added as a sensitivity analysis; QAT-5 remains the pre-registered result. (iv) The 25 % int8 models use the RAM objective.

IV. RESULTS

A. Accuracy and hardware cost

Table I gives accuracy for all 48 models and Table II the measured cost of each architecture. Int8 cuts latency 3.4–4.3× at every pruning ratio and Flash 1.9–3.4×; the Flash saving shrinks for small models because the runtime code is a fixed cost. The smallest model, int8 at 75 %, needs 79 KB of Flash and 35 KB of RAM, against 1,591 KB and 273 KB for the baseline. Int8 costs 1.6–1.8 pp of accuracy at 0–25 % pruning but 6.6 pp at 50 % and 15.0 pp at 75 % (PTQ).

B. Pareto front and selection

Fig. 1 plots accuracy against energy. Among the pre-registered variants, the front consists of the fp32 baseline, int8 PTQ at 0 %, and int8 QAT-5 at 25, 50 and 75 %. No compressed configuration meets the 1 pp rule, because int8 at 0 % already loses 1.61 pp; by the rule, the uncompressed baseline is selected. Table III shows the best choice at looser budgets: int8 alone saves 4.2× energy within 2 pp, and adding 25 % pruning saves 5.1× within 5 pp. Including QAT-15 picks the same architectures with 0.1–1.5 pp higher accuracy.

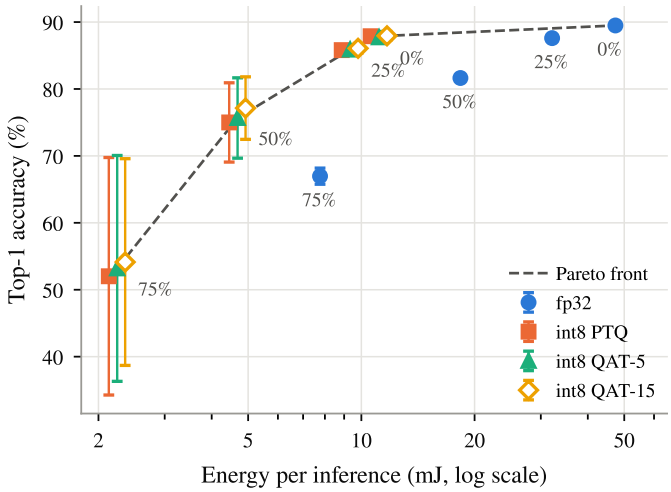


Fig. 1. Accuracy vs. estimated energy per inference (mean ± std, 3 seeds; labels give the pruning ratio). Int8 variants share one energy value per ratio and are offset horizontally for legibility. The dashed line is the Pareto front of the pre-registered variants.

TABLE III
LOWEST-ENERGY CONFIGURATION WITHIN AN ACCURACY BUDGET (PRE-REGISTERED VARIANTS)

Budget	Pruned	Variant	Acc. (%)	Energy (mJ)	Saving
1 pp	0 %	fp32	89.49	47.36	1.0×
2 pp	0 %	int8 PTQ	87.88	11.14	4.2×
5 pp	25 %	int8 QAT-5	85.88	9.33	5.1×
15 pp	50 %	int8 QAT-5	75.66	4.69	10.1×

C. H1: combined compression

All three pruning-only (fp32) points lie off the front: two are dominated by combined configurations and the 25 % point by int8 alone. For example, fp32 at 75 % reaches 66.97 % at 7.77 mJ, while int8 at 50 % reaches 75.66 % at 4.69 mJ. Combined configurations extend the front below 11 mJ but do not dominate int8 at 0 %, which remains the most accurate low-energy point. H1 therefore holds against pruning alone but not against quantization alone.

D. H2: QAT versus PTQ

Fig. 2 shows the accuracy gain of QAT over PTQ for each model. QAT-5 beats PTQ in 9 of 12 pairs by +0.43 pp on average (95 % CI $[-0.03, +0.90]$; Wilcoxon $p = 0.026$), and the gain grows with pruning ($\rho = 0.84$, $p = 0.001$): from −0.17 pp at 0 % to +1.17 pp at 75 %. QAT-15 raises the mean gain to +1.15 pp (CI $[+0.16, +2.14]$; 11 of 12 positive; $p < 0.001$), reaching +2.1 pp at 50 and 75 %. QAT-15 beats QAT-5 by +0.72 pp (CI $[+0.07, +1.36]$, $p = 0.001$). With three seeds the per-ratio intervals all include zero; the 12-pair test carries the evidence. H2 is supported, but QAT does not rescue the two most degraded models (next subsection).

E. H3: MACs as a latency proxy

Across the eight architectures, MACs and latency correlate with $\rho = 0.73$ ($p = 0.039$), below the 0.8 threshold, so H3

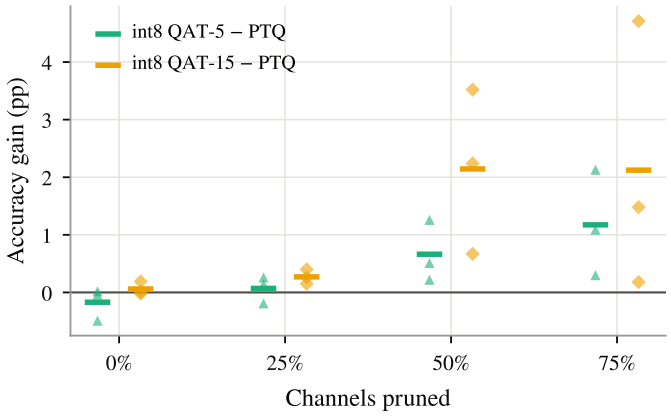


Fig. 2. Accuracy gain of QAT over PTQ, paired by model. Markers: individual seeds; bars: means.

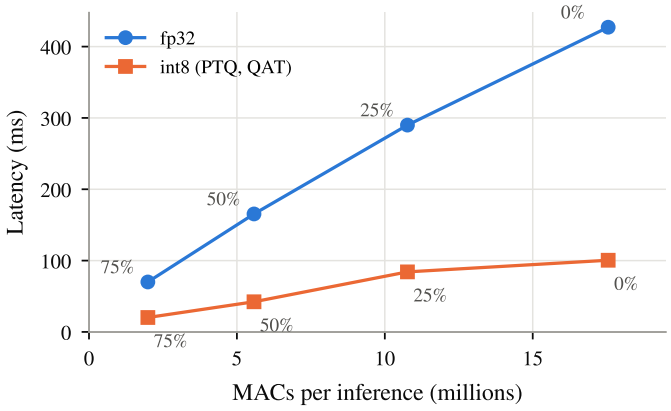


Fig. 3. MAC count vs. measured latency. Labels give the pruning ratio.

is not supported. Within one precision the ranking is perfect ($\rho = 1.0$ for fp32 and for int8; Fig. 3), but across precisions it fails: int8 at 0 % performs 17.6 M MACs in 100.5 ms, whereas fp32 at 50 % performs only 5.6 M MACs and takes 165.4 ms. Cost per MAC also rises as layers narrow, from 24.3 to 35.2 ns for fp32 and from 5.7 to 10.2 ns for int8, so pruning 25 % of int8 channels removes 39 % of MACs but only 16 % of latency.

F. Seed sensitivity

Fig. 4 shows every model. Full-precision accuracy varies by at most 2.3 pp across seeds, but int8 accuracy after aggressive pruning varies widely: 68.2–79.2 % at 50 % (PTQ) and 31.7–64.4 % at 75 %. The spread comes from two models. At 75 % with seed 1, int8 falls from 66.0 % to 31.7 % (PTQ), 33.8 % (QAT-5) and 36.4 % (QAT-15), while seed 3 loses about 2 pp. At 50 % with seed 2, accuracy falls from 81.7 % to 68.2–71.8 %. No calibration method rescues them: in the validation study their best-case drops were 25.7 pp and 12.5 pp. The sensitivity belongs to these model instances, not to the pipeline, which is identical across seeds.

V. Discussion

Practical guidance. For this model and device, int8 quantization should come first: it is the largest and cheapest saving,

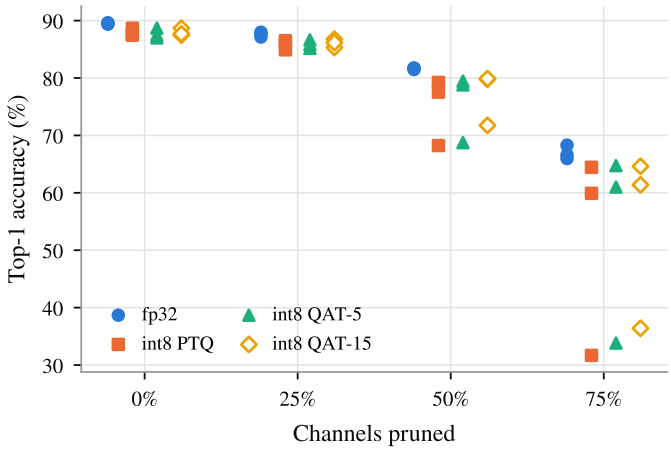


Fig. 4. Accuracy of all 48 models, one marker per seed.

4.2× energy and 3.3× Flash for 1.6 pp. Structured pruning adds less. The network is already narrow at width 0.35, so accuracy drops quickly, and the measured speed-up lags the reduction in MACs because narrow layers use the kernels less efficiently. Pruning 25 % on top of int8 is a reasonable step; going further trades accuracy steeply for modest additional savings.

Reporting. Two results bear on how compression results should be reported. First, MAC counts are a valid proxy only within one numeric precision; any comparison across fp32 and int8 needs measurement on the target. Second, a single seed could have reported 64 % or 32 % for the same 75 % int8 configuration. Studies of aggressive compression should report several seeds and the accuracy of every deployed model, not only a mean or the best run [26]. This supports the call in our review for mandatory, rigorous accuracy reporting alongside efficiency gains.

Why some models collapse. We did not identify the layer responsible. A plausible cause is the per-tensor activation quantization of depthwise layers whose channel ranges differ widely, a known weakness of MobileNet-style networks [13], [16] that pruning may amplify. Per-layer error analysis and cross-layer equalization [16] are natural next steps.

Limitations. The study covers one model, one dataset and one microcontroller; CIFAR-10 and a small MobileNetV2 limit how far the conclusions generalize. Energy is estimated from a datasheet figure for the microcontroller alone, not measured, and therefore mirrors latency; board-level power measurement would capture memory-access effects. Three seeds give wide per-ratio intervals. The 25 % int8 models used a different code-generation objective, the ST runtime chose memory placement per model, and one model showed a small numerical mismatch between board and reference. Accuracy was measured with ONNX Runtime rather than on the board.

VI. Conclusion

Under one fixed protocol on an STM32H7B3 microcontroller, int8 quantization dominated structured pruning alone,

saving 4.2× estimated energy for 1.6 pp of accuracy. Combining it with 25 % pruning raised the saving to 5.1× for 3.6 pp, but no compressed configuration met a 1 pp budget. QAT recovered a small and significant amount of accuracy over PTQ, growing with the pruning ratio. MAC counts failed as a latency proxy across precisions, and int8 accuracy after aggressive pruning varied by up to 33 pp between seeds. Compression studies for microcontrollers should therefore measure on the target and report results over several seeds. Code, configurations, raw device logs and results are available at [12].

REFERENCES

[1] M. Verhelst and B. Murmann, "Machine learning at the edge," in *NANO-CHIPS 2030*. Springer, 2020.

[2] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *International Conference on Learning Representations (ICLR)*, 2016.

[3] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[4] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.

[5] C. Hao, Y. Chen, X. Zhang, Y. Li, J. Xiong, W.-m. Hwu, and D. Chen, "Effective algorithm-accelerator co-design for AI solutions on edge devices," in *Great Lakes Symposium on VLSI (GLSVLSI)*, 2020.

[6] Y. Yang, Q. Huang, B. Wu *et al.*, "Synetgy: Algorithm-hardware co-design for ConvNet accelerators on embedded FPGAs," in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2019.

[7] F. N. Peccia, S. Pavlitska, T. Fleck, and O. Bringmann, "Efficient edge AI: Deploying convolutional neural networks on FPGA with the Gemmini accelerator," arXiv:2408.07404, 2024.

[8] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?" in *Proceedings of Machine Learning and Systems (MLSys)*, 2020.

[9] P. Chung, C. Chen, W. Chueh, K. Hsu, P. Hsieh, and Y. Fan, "Depthwise convolution with Winograd algorithm for efficient CNN acceleration," in *IEEE Gaming, Entertainment, and Media Conference (GEM)*, 2025.

[10] K. Choi and G. E. Sobelman, "An efficient CNN accelerator for low-cost edge systems," *ACM Transactions on Embedded Computing Systems*, vol. 21, no. 4, 2022.

[11] T.-J. Yang, Y.-H. Chen, and V. Sze, "Designing energy-efficient convolutional neural networks using energy-aware pruning," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[12] Y. Bekzat, "Quantization and pruning trade-offs for embedded CNN inference: Code, logs and results," https://github.com/bekadiscoteka/cnn-compression-study, 2026.

[13] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," arXiv:1806.08342, 2018.

[14] M. Rusci, A. Capotondi, and L. Benini, "Memory-driven mixed low precision quantization for enabling deep network inference on microcontrollers," in *Proceedings of Machine Learning and Systems (MLSys)*, 2020.

[15] A. Garofalo, G. Tagliavini, F. Conti, D. Rossi, and L. Benini, "XpulpNN: Accelerating quantized neural networks on RISC-V processors through ISA extensions," in *Design, Automation and Test in Europe Conference (DATE)*, 2020.

[16] M. Nagel, M. van Baalen, T. Blankevoort, and M. Welling, "Data-free quantization through weight equalization and bias correction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1325–1334.

[17] P. Morì, L. Frickenstein, S. B. Sampath *et al.*, "Wino vidi vici: Conquering numerical instability of 8-bit Winograd convolution for accurate inference acceleration on edge," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024.

[18] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[19] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," in *International Conference on Learning Representations (ICLR)*, 2017.

[20] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "DepGraph: Towards any structural pruning," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 16 091–16 101.

[21] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs," arXiv:1801.06601, 2018.

[22] C. Banbury, V. J. Reddi, P. Torelli *et al.*, "MLPerf Tiny benchmark," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[23] B. Moons, B. De Brabandere, L. Van Gool, and M. Verhelst, "Energy-efficient ConvNets through approximate computing," in *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2016.

[24] N. K. Jayakodi, A. Chatterjee, W. Choi, J. R. Doppa, and P. P. Pande, "Trading-off accuracy and energy of deep inference on embedded systems: A co-design approach," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 11, pp. 2881–2893, 2018.

[25] M. Hemmat, J. San Miguel, and A. Davoodi, "AirNN: A featherweight framework for dynamic input-dependent approximation of CNNs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 10, pp. 2090–2103, 2021.

[26] X. Bouthillier, P. Delaunay, M. Bronzi *et al.*, "Accounting for variance in machine learning benchmarks," in *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

[27] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[28] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[29] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[30] ONNX Runtime developers, "ONNX Runtime," https://onnxruntime.ai, 2025, version 1.29.

[31] STMicroelectronics, "ST Edge AI Developer Cloud," https://stedgeai-dc.st.com, 2026, ST Edge AI Core 4.1.0, accessed Oct. 2026.

[32] ——, "STM32H7A3/7B3 product overview," https://www.st.com/en/microcontrollers-microprocessors/stm32h7a3-7b3.html, 2026, accessed Oct. 2026.